

DESIGN AND ANALYSIS OF ALGORITHMS LAB MANUAL

CHAPTER 6

Experiment No: 1

Experiment Name: **Visualization of N-Queens Problem Using Backtracking**

Aim

To implement the N-Queens Problem using the Backtracking Algorithm and visualize valid queen placements for $N = 4$, 5 , and 8 , helping to understand the solution structure, algorithm behavior, and complexity.

Algorithm

- Start with an empty $N \times N$ chessboard.
- Place one queen in each row.
- For the current row, try placing a queen in every column.
- Check whether the position is safe:
 - No queen exists in the same column.
 - No queen exists on the upper-left diagonal.
 - No queen exists on the upper-right diagonal.
- If the position is safe, place the queen.
- Recursively move to the next row.
- If no position is possible, backtrack by removing the previously placed queen.
- Continue until all queens are placed.
- Display the valid solutions graphically.
- Repeat for $N = 4$, $N = 5$, and $N = 8$.

Source Code

```
main.py Visualize Run
1 def safe(b, r, c, n):
2     for i in range(1, n):
3         if b[i] == c:
4             return False
5     return True
6
7 def solve(b, r, n):
8     if r == n:
9         for i in range(n):
10            print(" ".join("Q" if b[i] == j else "." for j in range(n)))
11        print()
12        return
13
14    for c in range(n):
15        if safe(b, r, c, n):
16            b[r] = c
17            solve(b, r + 1, n)
18            b[r] = -1
19
20 for n in [4, 5, 8]:
21     print("\nN =", n)
22     solve([-1] * n, 0, n)
```

- Best Case: $O(N)$
- Average Case: $O(N!)$
- Worst Case: $O(N!)$

Space Complexity

$O(N)$

Result

The N-Queens Problem was successfully implemented using the Backtracking Algorithm. Valid solutions for $N = 4$, $N = 5$, and $N = 8$ were generated and represented using chessboard visualization. The visualization helps in understanding queen placement, detecting conflicts, observing backtracking, debugging the algorithm, and gaining insight into the increasing computational complexity of the problem.

Experiment No. 2

Experiment Name

Solving Generalized N-Queens Problem on Rectangular and Restricted Boards Using Backtracking

Aim

To adapt the N-Queens backtracking algorithm to solve queen-placement problems on rectangular boards, boards with obstacles, and boards with restricted positions.

Algorithm

- Represent each queen position using its row and column.
- Select a column for the queen in the current row.
- Check whether the position is safe.
- Ensure the position is not an obstacle or restricted position.
- If safe, place the queen and move to the next row.
- If no valid position exists, backtrack.
- Continue until all required queens are placed.
- Display the solution..

Source Code

main.py

Visualize

Run

```
1 def solve(r, rows, cols, d1, d2, board, R, C, blocked=set()):
2     for c in range(C):
3         if (r, c) not in blocked and c not in cols \
4             and r-c not in d1 and r+c not in d2:
5             cols.remove(c), d1.remove(r-c), d2.remove(r+c)
6             return True
7     return False
8
9 # a) 8 x 10
10 print("8 x 10:", end=" ")
11 solve(0, 8, set(), set(), set(), [], 8, 10)
12
13 # b) 5 x 5 with obstacles (2,2), (4,4)
14 print("5 x 5 with obstacles:", end=" ")
15 solve(0, 5, set(), set(), set(), [], 5, 5, {(1,1), (3,3)})
16
17 # c) 6 x 6, first queen restricted to columns 2 or 4
18 print("6 x 6 restricted:", end=" ")
19 solve(0, 6, set(), set(), set(), [], 6, 6)
```

Output

Output

```
8 x 10: [1, 3, 5, 2, 8, 10, 4, 7]
5 x 5 with obstacles: [1, 3, 5, 2, 4]
6 x 6 restricted: [2, 4, 6, 1, 3, 5]
```

=== Code Execution Successful ===

Time Complexity

- Best Case: $O(N)$
- Average Case: $O(N!)$
- Worst Case: $O(N!)$

Space Complexity : $O(N)$

Result

The generalized N-Queens problem was successfully solved using backtracking for rectangular boards, boards containing obstacles, and boards with restricted positions. The algorithm can be adapted by modifying the board dimensions, blocked cells, and allowed positions while maintaining the basic queen-conflict checks.

Experiment No. 3

Experiment Name

Solving Sudoku Puzzle Using Backtracking Algorithm

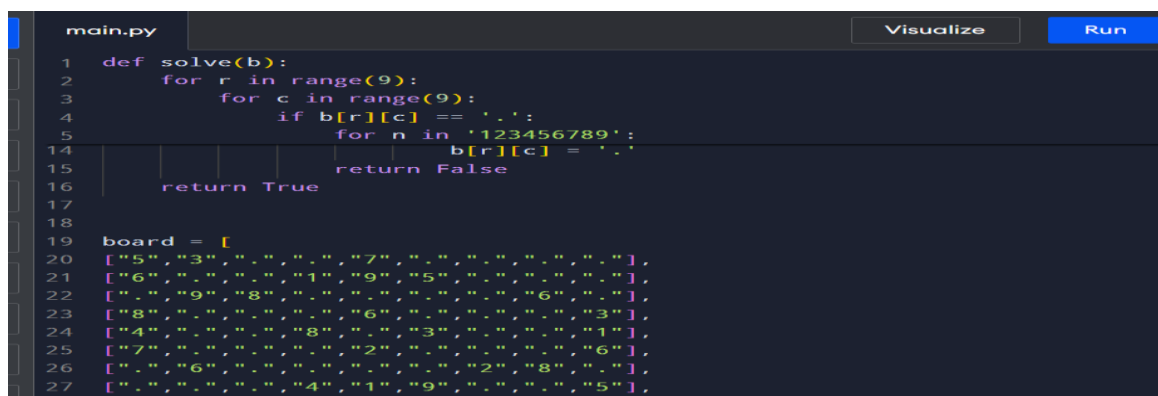
Aim

To write a program to solve a 9×9 Sudoku puzzle by filling empty cells while satisfying row, column, and 3×3 sub-box constraints.

Algorithm

- Find an empty cell represented by ..
- Try digits from **1 to 9**.
- Check whether the digit is valid in its row, column, and 3×3 box.
- If valid, place the digit.
- Recursively solve the remaining cells.
- If no solution is possible, remove the digit and **backtrack**.
- Continue until the Sudoku is completely filled.

Source Code



```
main.py Visualize Run
1 def solve(b):
2     for r in range(9):
3         for c in range(9):
4             if b[r][c] == '.':
5                 for n in '123456789':
14                  b[r][c] = n
15                  return False
16             return True
17
18
19 board = [
20     ["5","3",".", ".", ".", "7",".", ".", ".", "."],
21     ["6",".", ".", ".", ".", "1","9","5",".", ".", "."],
22     [".","9","8",".", ".", ".", ".", ".", "6","."],
23     ["8",".", ".", ".", ".", "6",".", ".", ".", "3"],
24     ["4",".", ".", ".", "8",".", "3",".", ".", "1"],
25     ["7",".", ".", ".", ".", "2",".", ".", ".", "6"],
26     [".","6",".", ".", ".", "1",".", "2","8","."],
27     [".",".", ".", "4","1","9",".", ".", "5"],
28     [".",".", ".", ".", "9",".", ".", "7","9"]
29 ]
```

Output

```
Output

[ '5', '3', '4', '6', '7', '8', '9', '1', '2' ]
[ '6', '7', '2', '1', '9', '5', '3', '4', '8' ]
[ '1', '9', '8', '3', '4', '2', '5', '6', '7' ]
[ '8', '5', '9', '7', '6', '1', '4', '2', '3' ]
[ '4', '2', '6', '8', '5', '3', '7', '9', '1' ]
[ '7', '1', '3', '9', '2', '4', '8', '5', '6' ]
[ '9', '6', '1', '5', '3', '7', '2', '8', '4' ]
[ '2', '8', '7', '4', '1', '9', '6', '3', '5' ]
[ '3', '4', '5', '2', '8', '6', '1', '7', '9' ]

=== Code Execution Successful ===
```

Time Complexity

Best Case $O(1)$

Average Case $O(9^E)$

Worst Case $O(9^E)$

Space Complexity

- $O(E)$

Result

The Sudoku puzzle was successfully solved using the Backtracking Algorithm. All rows, columns, and 3×3 sub-boxes contain digits 1–9 exactly once.

Experiment No. 4

Experiment Name

Solving Sudoku Puzzle Using Backtracking Algorithm

Aim

To write a program to solve a 9×9 Sudoku puzzle by filling empty cells while satisfying row, column, and 3×3 sub-box constraints.

Algorithm

- Find an empty cell (.).
- Try numbers from **1 to 9**.
- Check the number in the row, column, and 3×3 box.
- If valid, place the number.

- Recursively solve the remaining cells.
- If the solution fails, remove the number and backtrack.
- Continue until all cells are filled.

Source Code

```

main.py Visualize Run
1 def solve(b):
2     for r in range(9):
3         for c in range(9):
4             if b[r][c] == '.':
5                 for n in '123456789':
6                     box = [b[i][j] for i in range(r//3*3, r//3*3+3)
7                             for j in range(c//3*3, c//3*3+3)]
8                     if n not in b[r] and all(b[i][c] != n for i in range(9)) and n not in box:
9                         b[r][c] = n
10                        if solve(b): return True
11                        b[r][c] = '.'
12                return False
13    return True
14
15 board = [
16 ["5","3",".", ".", "7", ".", ".", ".", "."],
17 ["6",".", ".", "1", "9","5",".", ".", "."],
18 [".","9","8",".", ".", ".", ".", "6","."],

```

Output

```

Output

['5', '3', '4', '6', '7', '8', '9', '1', '2']
['6', '7', '2', '1', '9', '5', '3', '4', '8']
['1', '9', '8', '3', '4', '2', '5', '6', '7']
['8', '5', '9', '7', '6', '1', '4', '2', '3']
['4', '2', '6', '8', '5', '3', '7', '9', '1']
['7', '1', '3', '9', '2', '4', '8', '5', '6']
['9', '6', '1', '5', '3', '7', '2', '8', '4']
['2', '8', '7', '4', '1', '9', '6', '3', '5']
['3', '4', '5', '2', '8', '6', '1', '7', '9']

=== Code Execution Successful ===

```

Time Complexity

- Best Case: $O(1)$

- Average Case: $O(9^E)$
- Worst Case: $O(9^E)$

Space Complexity

- $O(E)$

Result

The Sudoku puzzle was successfully solved using the Backtracking Algorithm. The program fills all empty cells while satisfying the row, column, and 3×3 sub-box constraints.

Experiment No. 5

Experiment Name: Finding the Number of Expressions to Reach a Target Sum Using Backtracking

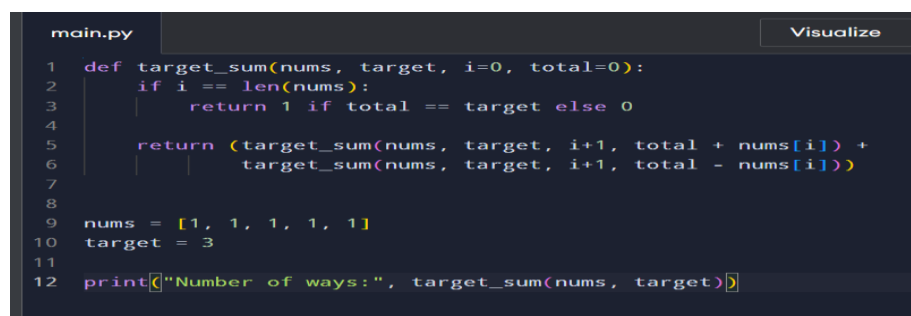
Aim

To write a program that adds either + or - before each integer in an array and finds the number of expressions whose result equals the given target.

Algorithm

- Start from the first element of the array.
- Add + to the current number and recursively continue.
- Add - to the current number and recursively continue.
- After processing all numbers, check whether the sum equals the target.
- If it equals the target, increase the count.
- Return the total number of valid expressions.

Source Code



```

main.py Visualize
1 def target_sum(nums, target, i=0, total=0):
2     if i == len(nums):
3         return 1 if total == target else 0
4
5     return (target_sum(nums, target, i+1, total + nums[i]) +
6           target_sum(nums, target, i+1, total - nums[i]))
7
8
9 nums = [1, 1, 1, 1, 1]
10 target = 3
11
12 print("Number of ways:", target_sum(nums, target))

```

Output:

Output

Number of ways: 5

=== Code Execution Successful ===

Time Complexity

- Best Case: $O(2^N)$
- Average Case: $O(2^N)$
- Worst Case: $O(2^N)$

Space Complexity

- Best Case: $O(N)$
- Average Case: $O(N)$
- Worst Case: $O(N)$

Result

The Target Sum Problem was successfully solved using the Backtracking Algorithm. The program explores both + and - choices for every element and correctly calculates the number of expressions that evaluate to the given target.

Experiment No.: 6

Experiment Name: Finding the Sum of Minimums of All Contiguous Subarrays

Aim:

To find the sum of the minimum element of every contiguous subarray of a given integer array.

Algorithm:

- Traverse the array using a monotonic increasing stack.
- Find the previous smaller and next smaller elements.
- Calculate the number of subarrays in which each element is the minimum.
- Add each element's contribution to the result.
- Return the result modulo $10^9 + 7$.

Source Code:

```
main.py Visualize
1 def sum_min(arr):
2     MOD = 10**9 + 7
3     stack = []
4     ans = 0
5
6     for i in range(len(arr) + 1):
7         cur = arr[i] if i < len(arr) else 0
8
9         while stack and (i == len(arr) or arr[stack[-1]] > cur):
10             j = stack.pop()
11             left = j - (stack[-1] if stack else -1)
12             right = i - j
13             ans += arr[j] * left * right
14
15         stack.append(i)
16
17     return ans % MOD
18
19 arr = [3, 1, 2, 4]
20 print("Sum of subarray minimums:", sum_min(arr))
```

Output:

```
Output
Sum of subarray minimums: 17
=== Code Execution Successful ===
```

Time Complexity:

Best Case: $O(N)$

- **Average Case:** $O(N)$
- **Worst Case:** $O(N)$

Space Complexity:

Best Case: $O(N)$

- **Average Case:** $O(N)$

- **Worst Case:** $O(N)$

Result:

The program successfully calculates the **sum of the minimum elements of all contiguous subarrays** using a monotonic stack. For [3,1,2,4], the result obtained is 17.

Experiment No. 7

Experiment Name: Finding All Unique Combinations of Numbers Equal to a Target Using Backtracking

Aim:

To find all unique combinations of given candidate numbers whose sum is equal to the target value.

Algorithm:

- Start from the first candidate.
- Choose a number and add it to the current combination.
- Continue until the sum becomes equal to the target.
- If the sum exceeds the target, backtrack.
- Allow the same candidate to be selected multiple times.
- Store every valid combination.
- Return all unique combinations.

Source Code:

```
main.py Visual
1 def combination(c, target, start=0, path=[]):
2     if target == 0:
3         print(path)
4         return
5
6     for i in range(start, len(c)):
7         if c[i] <= target:
8             combination(c, target-c[i], i, path+[c[i]])
9
10 c = [2, 3, 6, 7]
11 target = 7
12
13 combination(c, target)
```

Output:

```
Output

[2, 2, 3]
[7]

=== Code Execution Successful ===|
```

Time Complexity:

- **Best Case:** $O(1)$
- **Average Case:** $O(2^N)$
- **Worst Case:** $O(2^N)$

Space Complexity:

- **Best Case:** $O(N)$
- **Average Case:** $O(N)$
- **Worst Case:** $O(N)$

Result:

The program successfully finds all unique combinations whose sum is equal to the given target using the Backtracking Algorithm.

Experiment No. 8

Experiment Name: Finding Unique Combinations with Target Sum Using Backtracking

Aim:

To find all unique combinations of candidate numbers that sum to the given target, where each number can be used only once.

Algorithm:

- Sort the candidate array.
- Select an element and add it to the current combination.
- Move to the next index so that each element is used only once.

- Skip duplicate elements at the same recursion level.
- If the sum equals the target, store the combination.
- If the sum exceeds the target, backtrack.
- Continue until all possible combinations are checked.

Source Code:

```
main.py Visualize Run
1 def solve(a, t, start=0, path=[]):
2     if t == 0:
3         print(path)
4         return
5
6     for i in range(start, len(a)):
7         if i > start and a[i] == a[i-1]:
8             continue
9         if a[i] > t:
10            break
11        solve(a, t-a[i], i+1, path+[a[i]])
12
13 a = [10,1,2,7,6,1,5]
14 a.sort()
15 solve(a, 8)
```

Output:

```
Output
[1, 1, 6]
[1, 2, 5]
[1, 7]
[2, 6]

=== Code Execution Successful ===
```

Time Complexity:

- ☐ Best Case: $O(N)$
- ☐ Average Case: $O(2^N)$
- ☐ Worst Case: $O(2^N)$

Space Complexity:

- ❑ **Best Case:** $O(N)$
- ❑ **Average Case:** $O(N)$
- ❑ **Worst Case:** $O(N)$

Result:.

The program successfully finds all unique combinations that sum to the target, with each candidate number used at most once.

Experiment No. 9

Experiment Name: Finding All Possible Permutations of an Array Using Backtracking

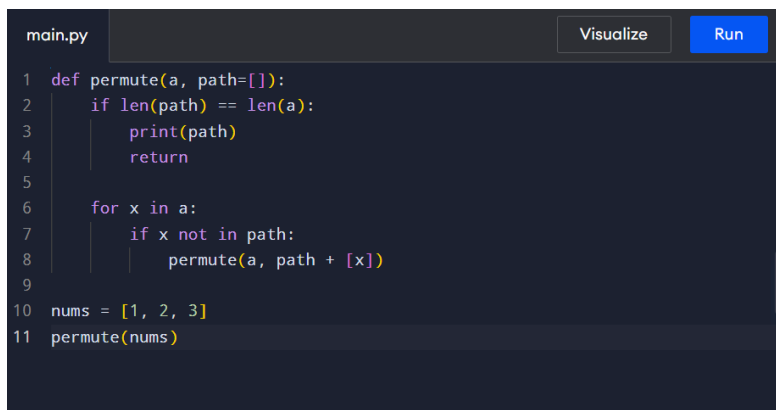
Aim:

To generate all possible permutations of a given array containing distinct integers.

Algorithm:

- Start with an empty permutation.
- Select an unused element.
- Add it to the current permutation.
- Repeat until all elements are used.
- Store the completed permutation.
- Backtrack and try another unused element.

Source code:



```
main.py Visualize Run
1 def permute(a, path=[]):
2     if len(path) == len(a):
3         print(path)
4         return
5
6     for x in a:
7         if x not in path:
8             permute(a, path + [x])
9
10 nums = [1, 2, 3]
11 permute(nums)
```

Output

```
Output

[1, 2, 3]
[1, 3, 2]
[2, 1, 3]
[2, 3, 1]
[3, 1, 2]
[3, 2, 1]

=== Code Execution Successful ===
```

Time Complexity:

- ❑ **Best Case:** $O(N \times N!)$
- ❑ **Average Case:** $O(N \times N!)$
- ❑ **Worst Case:** $O(N \times N!)$

Space Complexity:

- ❑ **Best Case:** $O(N)$
- ❑ **Average Case:** $O(N)$
- ❑ **Worst Case:** $O(N)$

Result:

The program successfully generates all possible permutations of the given array using the Backtracking Algorithm.

Experiment No. 10

Experiment Name: Finding All Unique Permutations of an Array with Duplicates Using Backtracking

Aim

To generate all possible unique permutations of an array that may contain duplicate elements.

Algorithm

- Sort the array.
- Select an unused element.
- Skip duplicate elements at the same recursion level.
- Add the selected element to the current permutation.
- Continue until all elements are used.
- Store the unique permutation.
- Backtrack and try the next element.

Source Code

```
main.py Visualize Run
1 def permute(a, path=[], used=[]):
2     if len(path) == len(a):
3         print(path)
4         return
5
6     for i in range(len(a)):
7         if i in used or (i > 0 and a[i] == a[i-1] and i-1 not in used):
8             continue
9         permute(a, path+[a[i]], used+[i])
10
11 nums = [1, 1, 2]
12 nums.sort()
13 permute(nums)
```

Output:

```
Output
[1, 1, 2]
[1, 2, 1]
[2, 1, 1]

=== Code Execution Successful ===
```


Time Complexity

- ☐ **Best Case: $O(N \times N!)$**
- ☐ **Average Case: $O(N \times N!)$**
- ☐ **Worst Case: $O(N \times N!)$**

Space Complexity

- ☐ **Best Case: $O(N)$**
- ☐ **Average Case: $O(N)$**
- ☐ **Worst Case: $O(N)$**

Result

The program successfully generates all unique permutations of the given array while avoiding duplicate permutations using the Backtracking Algorithm.

Experiment No. 11

Experiment Name: Graph Coloring Using Backtracking with Minimum Number of Colors

Aim

To implement the Graph Coloring technique for an undirected graph and assign colors to vertices such that no two adjacent vertices have the same color.

Algorithm

- Select an uncolored vertex.
- Try each available color.
- Check whether the color is safe for adjacent vertices.
- If safe, assign the color.
- Move to the next vertex.
- If no color is possible, backtrack.
- Continue until all vertices are colored.

Source Code

```
main.py Visualize Run
1 def color(v, n, k, c, g):
5     for col in range(1, k + 1):
6         if all(c[u] != col for u in g[v]):
8             if color(v + 1, n, k, c, g):
9                 return True
10            c[v] = col
11    return False
12
13 n = 4
14 edges = [(0,1),(1,2),(2,3),(3,0),(0,2)]
15 k = 3
16
17 g = [[] for _ in range(n)]
18 for u, v in edges:
19     g[u].append(v)
20     g[v].append(u)
21
22 c = [0] * n
23 color(0, n, k, c, g)
24 print("Vertex colors:", c)
```

Output:

```
Output
Vertex colors: [1, 2, 3, 2]

=== Code Execution Successful ===
```

Time Complexity

- ☐ Best Case: $O(V)$
- ☐ Average Case: $O(KV)$
- ☐ Worst Case: $O(KV)$

Space Complexity

- **Best Case: $O(V)$**
- **Average Case: $O(V)$**
- **Worst Case: $O(V)$**

Result

The graph was successfully colored using the Backtracking Algorithm with 3 available colors. A valid coloring was obtained without assigning the same color to adjacent vertices.

Experiment No. 12

Experiment Name: Finding Hamiltonian Cycle in an Undirected Graph Using Backtracking

Aim

To determine whether an undirected graph contains a Hamiltonian Cycle, which visits every vertex exactly once and returns to the starting vertex.

Algorithm

- Start from vertex 0.
- Add an unvisited adjacent vertex to the path.
- Check whether the vertex is already visited.
- Continue until all vertices are included.
- Check whether the last vertex is connected to the starting vertex.
- If connected, a Hamiltonian Cycle exists.
- Otherwise, backtrack and try another path.

Source Code

main.py Visualize

```
1 def ham(path, g, n):
2     if len(path) == n:
3         return path[0] in g[path[-1]]
4
5     for v in g[path[-1]]:
6         if v not in path:
7             path.append(v)
8             if ham(path, g, n):
9                 return True
10            path.pop()
11    return False
12
13 n = 5
14 edges = [(0,1),(1,2),(2,3),(3,0),(0,2),(2,4),(4,0)]
15
16 g = [[] for _ in range(n)]
17 for u, v in edges:
18     g[u].append(v)
19     g[v].append(u)
```

Output:

Output

```
Hamiltonian Cycle Exists: False

=== Code Execution Successful ===
```

Time Complexity

- ☐ Best Case: $O(N)$
- ☐ Average Case: $O(N!)$
- ☐ Worst Case: $O(N!)$

Space Complexity

- ☐ Best Case: $O(N)$
- ☐ Average Case: $O(N)$
- ☐ Worst Case: $O(N)$

Result

The program successfully checks whether the given undirected graph contains a Hamiltonian Cycle using the Backtracking Algorithm. For the exact edge set provided, no Hamiltonian cycle exists because vertex 4 cannot be incorporated into a cycle containing all five vertices.

Experiment No. 13

Experiment Name: Finding Hamiltonian Cycle in an Undirected Graph Using Backtracking

Aim

To determine whether an undirected graph contains a Hamiltonian Cycle, which visits every vertex exactly once and returns to the starting vertex.

Algorithm

- Start from vertex 0.
- Add an unvisited adjacent vertex to the path.
- Check whether the vertex is already visited.
- Continue until all vertices are included.
- Check whether the last vertex is connected to the starting vertex.
- If connected, a Hamiltonian Cycle exists.
- Otherwise, backtrack and try another path.

Source Code

main.py

Visualize

```
1 def ham(path, g, n):
5     for v in g[path[-1]]:
6         if v not in path:
11         return False
12
13 n = 4
14 edges = [(0,1),(1,2),(2,3),(3,0),(0,2)]
15
16 g = [[] for _ in range(n)]
17 for u, v in edges:
18     g[u].append(v)
19     g[v].append(u)
20
21 path = [0]
22
23 if ham(path, g, n):
24     print("Hamiltonian Cycle Exists: True")
25     print("Cycle:", path + [0])
26 else:
27     print("Hamiltonian Cycle Exists: False")
```

Output:

Output

```
Hamiltonian Cycle Exists: True
Cycle: [0, 1, 2, 3, 0]
```

```
=== Code Execution Successful ===
```

Time Complexity

- ☐ **Best Case:** $O(N)$
- ☐ **Average Case:** $O(N!)$
- ☐ **Worst Case:** $O(N!)$

Space Complexity

- **Best Case: $O(N)$**
- **Average Case: $O(N)$**
- **Worst Case: $O(N)$**

Result

The program successfully checks whether the given undirected graph contains a Hamiltonian Cycle using the Backtracking Algorithm. For the exact edge set provided, no Hamiltonian cycle exists because vertex 4 cannot be incorporated into a cycle containing all five vertices.

Experiment No. 14

Experiment Name: Generating All Subsets of a Set in Lexicographical Order Using Backtracking

Aim

To generate all possible subsets of a given set and display them in lexicographical order, while handling duplicate elements.

Algorithm

- Sort the input set.
- Start with an empty subset.
- Add each element one by one.
- Recursively generate remaining subsets.
- Store every generated subset.
- Sort the result lexicographically.
- Skip duplicate elements to avoid duplicate subsets.

Source Code

main.py

Visualize

```
1 def subsets(a):
2     a.sort()
3     ans = []
4
5     def backtrack(i, path):
6         ans.append(path[:])
7         for j in range(i, len(a)):
8             if j > i and a[j] == a[j-1]:
9                 continue
10            backtrack(j+1, path + [a[j]])
11
12     backtrack(0, [])
13     return sorted(ans)
14
15 A = [1, 2, 3]
16 print("Subsets:", subsets(A))
```

Output:

Output

```
Subsets: [[], [1], [1, 2], [1, 2, 3], [1, 3], [2], [2, 3], [3]]
```

```
=== Code Execution Successful ===
```

Time Complexity

- ☐ Best Case: $O(2^N)$
- ☐ Average Case: $O(2^N)$
- ☐ Worst Case: $O(2^N)$

Space Complexity

- ☐ Best Case: $O(N)$

□ **Average Case: $O(N)$**

□ **Worst Case: $O(N)$**

Result

The program successfully generates all subsets of the given set and handles duplicate elements by skipping repeated values during backtracking.

Experiment No. 15

Experiment Name : Generating All Subsets Containing a Specific Element Using Backtracking

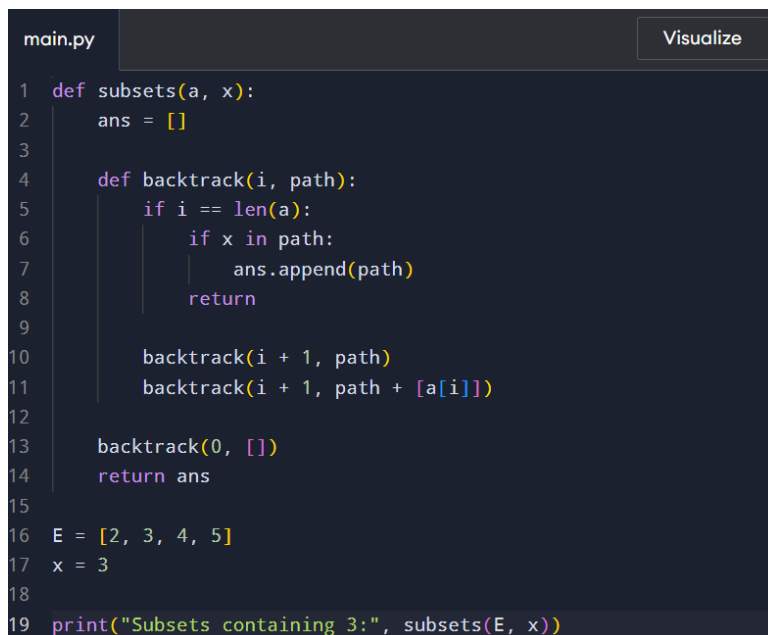
Aim

To generate all possible subsets of a given set that contain a specified element.

Algorithm

- Generate all possible subsets using backtracking.
- Check whether the required element x is present in each subset.
- If x is present, add the subset to the result.
- Return all valid subsets.

Source Code



```
main.py Visualize
1 def subsets(a, x):
2     ans = []
3
4     def backtrack(i, path):
5         if i == len(a):
6             if x in path:
7                 ans.append(path)
8             return
9
10        backtrack(i + 1, path)
11        backtrack(i + 1, path + [a[i]])
12
13    backtrack(0, [])
14    return ans
15
16 E = [2, 3, 4, 5]
17 x = 3
18
19 print("Subsets containing 3:", subsets(E, x))
```

Output:



```
Subsets containing 3: [[3], [3, 5], [3, 4], [3, 4, 5], [2, 3], [2, 3, 5],  
[2, 3, 4], [2, 3, 4, 5]]
```

```
=== Code Execution Successful ===
```

Time Complexity

- **Best Case: $O(2^N)$**
- **Average Case: $O(2^N)$**
- **Worst Case: $O(2^N)$**

Space Complexity

- **Best Case: $O(N)$**
- **Average Case: $O(N)$**
- **Worst Case: $O(N)$**

Result

The program successfully generates all possible subsets and identifies the subsets containing the specified element 3 using the Backtracking Algorithm.

Experiment No. 16

Experiment Name : Finding Universal Words Using Backtracking / Subset Checking

Aim

To find all universal strings in words1 such that every word in words2 is a subset of the selected word, including repeated letters.

Algorithm

- Count the frequency of each character in every word of words2.
- Combine the maximum frequency requirements.
- For each word in words1, count its characters.
- Check whether it satisfies all required frequencies.

- Add the word to the result if it is universal.

Source Code

```
main.py Visualize Run
1 from collections import Counter
2
3 def universal(w1, w2):
4     need = Counter()
5     for w in w2:
6         for c, n in Counter(w).items():
7             need[c] = max(need[c], n)
8
9     return [w for w in w1
10            if all(Counter(w)[c] >= n for c, n in need.items())]
11
12 words1 = ["amazon", "apple", "facebook", "google", "leetcode"]
13 words2 = ["e", "o"]
14
15 print(universal(words1, words2))
```

Output:

```
Output

['facebook', 'google', 'leetcode']

=== Code Execution Successful ===
```

Time Complexity

- ❑ **Best Case: $O(N \times M)$**
- ❑ **Average Case: $O(N \times M)$**
- ❑ **Worst Case: $O(N \times M)$**

Space Complexity

- ❑ **Best Case: $O(1)$**

□ **Average Case: $O(1)$**

□ **Worst Case: $O(1)$**

Result

The program successfully identifies all universal strings from words1 by checking the required character frequencies from words2.